

Prompt usado:

"Sistema de Análise e Otimização de Trelças Planas Isostáticas" Uma empresa de engenharia civil precisa de um software capaz de analisar trelças planas (estruturas formadas por barras conectadas em nós) de tamanhos variados. O usuário deve poder definir, a cada execução, o número de nós e barras da estrutura, suas coordenadas, conexões, cargas aplicadas e apoios. O programa deverá calcular as forças internas em cada barra (tração ou compressão) e gerar visualizações gráficas dos resultados. A solução deve ser implementada em Python, utilizando vetores/listas de tamanho variável, definidos a cada execução. Os resultados devem ser apresentados em forma de tabela e também por meio de gráficos visuais.

Chat GPT:

```
import numpy as np
import matplotlib.pyplot as plt
import pandas as pd

# =====
# DADOS DA TRELIÇA
# =====

# Coordenadas dos nós
nos = np.array([
    [0, 0],
    [4, 0],
    [2, 3]
])

# Barras (nó inicial, nó final)
barras = [
    (0, 1),
    (0, 2),
    (1, 2)
]

# Propriedades
E = 210e9      # Módulo de elasticidade (Pa)
A = 0.01       # Área da seção (m²)

# =====
# GRAUS DE LIBERDADE
# =====
```

```

num_nos = len(nos)
gl = 2 * num_nos

K_global = np.zeros((gl, gl))

# =====
# MONTAGEM DA MATRIZ GLOBAL
# =====

for barra in barras:

    i, j = barra

    xi, yi = nos[i]
    xj, yj = nos[j]

    L = np.sqrt((xj - xi)**2 + (yj - yi)**2)

    c = (xj - xi) / L
    s = (yj - yi) / L

    k = (E * A / L) * np.array([
        [ c*c,   c*s, -c*c, -c*s],
        [ c*s,   s*s, -c*s, -s*s],
        [-c*c, -c*s,  c*c,  c*s],
        [-c*s, -s*s,  c*s,  s*s]
    ])

    indices = [
        2*i,
        2*i + 1,
        2*j,
        2*j + 1
    ]

    for a in range(4):
        for b in range(4):
            K_global[indices[a], indices[b]] += k[a, b]

# =====
# FORÇAS EXTERNAS
# =====

```

```

F = np.zeros(gl)

# Carga vertical no nó 3
F[5] = -10000

# =====
# APOIOS
# =====

# Nó 1 fixo em X e Y
# Nó 2 fixo em Y

restritos = [0, 1, 3]

livres = []

for i in range(gl):
    if i not in restritos:
        livres.append(i)

K_reduzida = K_global[np.ix_(livres, livres)]
F_reduzida = F[livres]

# =====
# DESLOCAMENTOS
# =====

U = np.zeros(gl)

U_livres = np.linalg.solve(K_reduzida, F_reduzida)

for i in range(len(livres)):
    U[livres[i]] = U_livres[i]

# =====
# FORÇAS NAS BARRAS
# =====

forcas = []

for barra in barras:

    i, j = barra

```

```

xi, yi = nos[i]
xj, yj = nos[j]

L = np.sqrt((xj - xi)**2 + (yj - yi)**2)

c = (xj - xi) / L
s = (yj - yi) / L

u = np.array([
    U[2*i],
    U[2*i + 1],
    U[2*j],
    U[2*j + 1]
])

deformacao = (1 / L) * np.array([-c, -s, c, s]) @ u

forca = E * A * deformacao

forcas.append(forca)

# =====
# TABELA DE RESULTADOS
# =====

dados = []

for i, f in enumerate(forcas):

    tipo = "Tração"

    if f < 0:
        tipo = "Compressão"

    dados.append([
        i + 1,
        round(f, 2),
        tipo
    ])

df = pd.DataFrame(
    dados,

```

```

        columns=["Barra", "Força (N)", "Tipo"]
    )

print(df)

# =====
# PLOTAGEM
# =====

plt.figure(figsize=(8,6))

for i, barra in enumerate(barras):

    ni, nj = barra

    xi, yi = nos[ni]
    xj, yj = nos[nj]

    cor = 'blue'

    if forcas[i] < 0:
        cor = 'red'

    plt.plot(
        [xi, xj],
        [yi, yj],
        color=cor,
        linewidth=3
    )

    xm = (xi + xj) / 2
    ym = (yi + yj) / 2

    plt.text(
        xm,
        ym,
        f"{forcas[i]:.0f} N"
    )

plt.scatter(nos[:,0], nos[:,1], s=100)

for i, (x, y) in enumerate(nos):
    plt.text(x, y + 0.2, f"Nó {i+1}")

```

```
plt.title("Análise de Trelença")
plt.axis('equal')
plt.grid(True)

plt.show()
```

Claude IA:

```
<!DOCTYPE html>
<html>
<head>
<style>
* { box-sizing: border-box; margin: 0; padding: 0; }
body { font-family: 'Courier New', monospace; background: transparent;
color: var(--color-text-primary); font-size: 13px; }

.app { display: flex; flex-direction: column; gap: 0; min-height:
600px; }

.header { background: var(--color-background-secondary); border-bottom:
0.5px solid var(--color-border-secondary); padding: 12px 16px; display:
flex; align-items: center; gap: 10px; }
.header-title { font-size: 14px; font-weight: 500; letter-spacing:
0.05em; color: var(--color-text-primary); }
.header-sub { font-size: 11px; color: var(--color-text-tertiary); }
.badge { font-size: 10px; padding: 2px 8px; border-radius: 3px;
background: var(--color-background-info); color:
var(--color-text-info); font-weight: 500; border: 0.5px solid
var(--color-border-tertiary); }

.tabs { display: flex; border-bottom: 0.5px solid
var(--color-border-tertiary); background:
var(--color-background-primary); }
.tab { padding: 8px 16px; font-size: 12px; cursor: pointer;
border-bottom: 2px solid transparent; color:
var(--color-text-secondary); transition: all 0.15s; font-family:
inherit; background: none; border-left: none; border-right: none;
border-top: none; }
```

```
.tab:hover { color: var(--color-text-primary); background:
var(--color-background-secondary); }
.tab.active { color: var(--color-text-primary); border-bottom-color:
var(--color-text-primary); font-weight: 500; }

.content { display: flex; gap: 0; flex: 1; }
.panel { padding: 14px; flex: 1; overflow-y: auto; }
.panel-right { border-left: 0.5px solid var(--color-border-tertiary);
min-width: 260px; max-width: 260px; }

.section { margin-bottom: 16px; }
.section-label { font-size: 10px; font-weight: 500; color:
var(--color-text-tertiary); letter-spacing: 0.08em; text-transform:
uppercase; margin-bottom: 8px; padding-bottom: 4px; border-bottom:
0.5px solid var(--color-border-tertiary); }

.form-row { display: grid; gap: 6px; margin-bottom: 6px; }
.form-row.cols-2 { grid-template-columns: 1fr 1fr; }
.form-row.cols-3 { grid-template-columns: 1fr 1fr 1fr; }
.form-row.cols-4 { grid-template-columns: 1fr 1fr 1fr 1fr; }
.form-row.cols-5 { grid-template-columns: 1fr 1fr 1fr 1fr 1fr; }

label { font-size: 10px; color: var(--color-text-tertiary); display:
block; margin-bottom: 2px; }
input[type=number], input[type=text], select {
  width: 100%; padding: 5px 7px; font-size: 12px; font-family: inherit;
  background: var(--color-background-secondary); border: 0.5px solid
var(--color-border-secondary);
  border-radius: 4px; color: var(--color-text-primary); outline: none;
  transition: border-color 0.15s;
}
input:focus, select:focus { border-color: var(--color-border-primary);
}

.btn { padding: 6px 12px; font-size: 12px; font-family: inherit;
border-radius: 4px; cursor: pointer; border: 0.5px solid
var(--color-border-secondary); background:
var(--color-background-secondary); color: var(--color-text-primary);
transition: all 0.15s; font-weight: 500; }
.btn:hover { background: var(--color-background-tertiary); }
.btn-primary { background: var(--color-text-primary); color:
var(--color-background-primary); border-color:
var(--color-text-primary); }
```

```
.btn-primary:hover { opacity: 0.85; }
.btn-danger { border-color: var(--color-border-danger); color:
var(--color-text-danger); }
.btn-danger:hover { background: var(--color-background-danger); }
.btn-sm { padding: 3px 8px; font-size: 11px; }
.btn-group { display: flex; gap: 6px; flex-wrap: wrap; }

.data-table { width: 100%; border-collapse: collapse; font-size: 11px;
margin-bottom: 8px; }
.data-table th { text-align: left; padding: 4px 6px; font-size: 10px;
color: var(--color-text-tertiary); font-weight: 500; border-bottom:
0.5px solid var(--color-border-tertiary); letter-spacing: 0.05em; }
.data-table td { padding: 4px 6px; border-bottom: 0.5px solid
var(--color-border-tertiary); color: var(--color-text-primary);
vertical-align: middle; }
.data-table tr:last-child td { border-bottom: none; }
.data-table tr:hover td { background:
var(--color-background-secondary); }

.pill { display: inline-block; padding: 1px 6px; border-radius: 10px;
font-size: 10px; font-weight: 500; }
.pill-t { background: var(--color-background-info); color:
var(--color-text-info); }
.pill-c { background: var(--color-background-danger); color:
var(--color-text-danger); }
.pill-z { background: var(--color-background-secondary); color:
var(--color-text-tertiary); }

.stat-grid { display: grid; grid-template-columns: 1fr 1fr; gap: 8px;
margin-bottom: 12px; }
.stat-card { background: var(--color-background-secondary); border:
0.5px solid var(--color-border-tertiary); border-radius: 6px; padding:
10px; }
.stat-val { font-size: 18px; font-weight: 500; color:
var(--color-text-primary); }
.stat-lbl { font-size: 10px; color: var(--color-text-tertiary);
margin-top: 2px; }

.canvas-wrap { background: var(--color-background-secondary); border:
0.5px solid var(--color-border-tertiary); border-radius: 6px; overflow:
hidden; position: relative; }
canvas { display: block; }
```



```
.alert { padding: 8px 12px; border-radius: 4px; font-size: 11px;
margin-bottom: 8px; }
.alert-info { background: var(--color-background-info); color:
var(--color-text-info); border: 0.5px solid var(--color-border-info); }
.alert-danger { background: var(--color-background-danger); color:
var(--color-text-danger); border: 0.5px solid
var(--color-border-danger); }
.alert-success { background: var(--color-background-success); color:
var(--color-text-success); border: 0.5px solid
var(--color-border-success); }

.legend { display: flex; gap: 14px; flex-wrap: wrap; margin-top: 6px;
font-size: 10px; color: var(--color-text-secondary); }
.legend-item { display: flex; align-items: center; gap: 4px; }
.legend-dot { width: 10px; height: 3px; border-radius: 1px; }

.preset-grid { display: grid; grid-template-columns: 1fr 1fr; gap: 6px;
margin-bottom: 10px; }
.preset-btn { padding: 8px; font-size: 10px; font-family: inherit;
border-radius: 4px; cursor: pointer; border: 0.5px solid
var(--color-border-secondary); background:
var(--color-background-secondary); color: var(--color-text-secondary);
transition: all 0.15s; text-align: left; line-height: 1.4; }
.preset-btn:hover { border-color: var(--color-border-primary); color:
var(--color-text-primary); background:
var(--color-background-tertiary); }
.preset-name { font-weight: 500; font-size: 11px; display: block; }
.preset-desc { font-size: 9px; color: var(--color-text-tertiary); }

.tab-content { display: none; }
.tab-content.active { display: block; }

.num-strong { font-variant-numeric: tabular-nums; font-weight: 500; }

.tooltip-overlay { position: absolute; background:
var(--color-background-primary); border: 0.5px solid
var(--color-border-secondary); border-radius: 4px; padding: 6px 8px;
font-size: 10px; pointer-events: none; z-index: 10; display: none;
color: var(--color-text-primary); }
</style>
</head>
<body>
<div class="app">
```

```

<div class="header">
  <div>
    <div class="header-title">O SISTEMA DE ANÁLISE DE TRELIÇAS
PLANAS</div>
    <div class="header-sub">Método dos Nós – Estruturas
Isostáticas</div>
  </div>
  <span class="badge">Python / NumPy Engine</span>
</div>

<div class="tabs">
  <button class="tab active" onclick="showTab('entrada')">① Entrada
de Dados</button>
  <button class="tab" onclick="showTab('resultados')">②
Resultados</button>
  <button class="tab" onclick="showTab('visualizacao')">③
Visualização</button>
</div>

<!-- TAB 1: ENTRADA -->
<div id="tab-entrada" class="tab-content active">
  <div class="content">
    <div class="panel">
      <div class="section">
        <div class="section-label">Exemplos Predefinidos</div>
        <div class="preset-grid">
          <button class="preset-btn"
onclick="loadPreset('warren')"><span class="preset-name">Trelça
Warren</span><span class="preset-desc">4 nós, 5 barras</span></button>
          <button class="preset-btn"
onclick="loadPreset('pratt')"><span class="preset-name">Trelça
Pratt</span><span class="preset-desc">6 nós, 9 barras</span></button>
          <button class="preset-btn"
onclick="loadPreset('simples')"><span class="preset-name">Trelça
Simples</span><span class="preset-desc">3 nós, 3 barras</span></button>
          <button class="preset-btn"
onclick="loadPreset('ponte')"><span class="preset-name">Ponte em
Trelça</span><span class="preset-desc">8 nós, 13
barras</span></button>
        </div>
      </div>
    </div>

    <div class="section">

```

```

        <div class="section-label">Nós (Coordenadas em metros)</div>
        <div id="nos-container"></div>
        <div class="btn-group" style="margin-top:6px">
            <button class="btn btn-sm" onclick="addNo()">+ Adicionar
Nó</button>
            <button class="btn btn-sm btn-danger"
onclick="removeNo()">- Remover Último</button>
        </div>
    </div>

    <div class="section">
        <div class="section-label">Barras (Conexões entre Nós)</div>
        <div id="barras-container"></div>
        <div class="btn-group" style="margin-top:6px">
            <button class="btn btn-sm" onclick="addBarra()">+ Adicionar
Barra</button>
            <button class="btn btn-sm btn-danger"
onclick="removeBarra()">- Remover Última</button>
        </div>
    </div>

    <div class="panel panel-right">
        <div class="section">
            <div class="section-label">Apoios</div>
            <div id="apoios-container"></div>
            <div class="btn-group" style="margin-top:6px">
                <button class="btn btn-sm" onclick="addApoio()">+ Adicionar
Apoio</button>
                <button class="btn btn-sm btn-danger"
onclick="removeApoio()">- Remover Último</button>
            </div>
        </div>

        <div class="section">
            <div class="section-label">Cargas Externas (kN)</div>
            <div id="cargas-container"></div>
            <div class="btn-group" style="margin-top:6px">
                <button class="btn btn-sm" onclick="addCarga()">+ Adicionar
Carga</button>
                <button class="btn btn-sm btn-danger"
onclick="removeCarga()">- Remover Última</button>
            </div>
        </div>
    </div>

```

```

    </div>

    <div class="section">
        <button class="btn btn-primary" style="width:100%;
margin-top:4px" onclick="analisar()">▶ Analisar Estrutura</button>
    </div>

    <div id="status-msg"></div>
</div>
</div>
</div>

<!-- TAB 2: RESULTADOS -->
<div id="tab-resultados" class="tab-content">
    <div class="panel">
        <div id="results-content">
            <div class="alert alert-info">Execute a análise na aba "Entrada
de Dados" para ver os resultados.</div>
        </div>
    </div>
</div>

<!-- TAB 3: VISUALIZAÇÃO -->
<div id="tab-visualizacao" class="tab-content">
    <div class="panel">
        <div id="viz-content">
            <div class="alert alert-info">Execute a análise para visualizar
a estrutura.</div>
        </div>
    </div>
</div>
</div>

<script>
// === STATE ===
let nos = [], barras = [], apoios = [], cargas = [];
let resultados = null;

// === PRESETS ===
const presets = {
    simples: {
        nos: [{x:0,y:0},{x:4,y:0},{x:2,y:3}],
        barras: [{i:0,j:1},{i:0,j:2},{i:1,j:2}],

```

```

    apoios: [{no:0, tipo:'fixo'}, {no:1, tipo:'rolete_x'}],
    cargas: [{no:2, fx:0, fy:-20}]
  },
  warren: {
    nos: [{x:0, y:0}, {x:3, y:0}, {x:6, y:0}, {x:1.5, y:2.6}, {x:4.5, y:2.6}],
    barras:
    [{i:0, j:1}, {i:1, j:2}, {i:0, j:3}, {i:1, j:3}, {i:1, j:4}, {i:2, j:4}, {i:3, j:4}]
  },
  apoios: [{no:0, tipo:'fixo'}, {no:2, tipo:'rolete_x'}],
  cargas: [{no:3, fx:0, fy:-15}, {no:4, fx:0, fy:-15}]
},
pratt: {
  nos: [{x:0, y:0}, {x:3, y:0}, {x:6, y:0}, {x:9, y:0}, {x:3, y:3}, {x:6, y:3}],
  barras:
    [{i:0, j:1}, {i:1, j:2}, {i:2, j:3}, {i:0, j:4}, {i:1, j:4}, {i:1, j:5}, {i:2, j:5},
    {i:3, j:5}, {i:4, j:5}],
  apoios: [{no:0, tipo:'fixo'}, {no:3, tipo:'rolete_x'}],
  cargas: [{no:1, fx:0, fy:-20}, {no:2, fx:0, fy:-20}, {no:4, fx:5, fy:0}]
},
ponte: {
  nos:
    [{x:0, y:0}, {x:4, y:0}, {x:8, y:0}, {x:12, y:0}, {x:2, y:4}, {x:6, y:4}, {x:10, y:4},
    {x:4, y:0}],
  barras:
    [{i:0, j:1}, {i:1, j:2}, {i:2, j:3}, {i:0, j:4}, {i:4, j:5}, {i:5, j:6}, {i:6, j:3},
    {i:1, j:4}, {i:1, j:5}, {i:2, j:5}, {i:2, j:6}],
  apoios: [{no:0, tipo:'fixo'}, {no:3, tipo:'rolete_x'}],
  cargas: [{no:1, fx:0, fy:-25}, {no:2, fx:0, fy:-25}, {no:5, fx:0, fy:-10}]
}
};

```

```

function loadPreset(name) {
  const p = presets[name];
  nos = p.nos.map(n => ({...n}));
  barras = p.barras.map(b => ({...b}));
  apoios = p.apoios.map(a => ({...a}));
  cargas = p.cargas.map(c => ({...c}));
  renderAll();
}

```

```

// === RENDER INPUTS ===

```

```

function renderAll() {
  renderNos(); renderBarras(); renderApoios(); renderCargas();
}

```

```
}
```

```
function renderNos() {
```

```
  const c = document.getElementById('nos-container');
```

```
  let h = `



(m)</th><th>Y (m)</th></tr></thead><tbody>`;
```

```
  nos.forEach((n,i) => {
```

```
    h += `
```

```
      <td style="color:var(--color-text-tertiary)">N${i}</td>
```

```
      <td><input type="number" step="0.1" value="${n.x}"  
onchange="nos[${i}].x+=this.value" style="padding:3px 5px"></td>
```

```
      <td><input type="number" step="0.1" value="${n.y}"  
onchange="nos[${i}].y+=this.value" style="padding:3px 5px"></td>
```

```
    </tr>`;
```

```
  });
```

```
  h += `</tbody></table>`;
```

```
  c.innerHTML = h;
```

```
}
```

```
function renderBarras() {
```

```
  const c = document.getElementById('barras-container');
```

```
  let h = `



i</th><th>Nó j</th></tr></thead><tbody>`;
```

```
  barras.forEach((b,k) => {
```

```
    let opts_i = nos.map((_,i)=>`<option value="${i}"  
${b.i===i?'selected':''}>N${i}</option>`).join('');
```

```
    let opts_j = nos.map((_,j)=>`<option value="${j}"  
${b.j===j?'selected':''}>N${j}</option>`).join('');
```

```
    h += `
```

```
      <td style="color:var(--color-text-tertiary)">B${k}</td>
```

```
      <td><select onchange="barras[${k}].i+=this.value"  
style="padding:3px 5px">${opts_i}</select></td>
```

```
      <td><select onchange="barras[${k}].j+=this.value"  
style="padding:3px 5px">${opts_j}</select></td>
```

```
    </tr>`;
```

```
  });
```

```
  h += `</tbody></table>`;
```

```
  c.innerHTML = h;
```

```
}
```

```
function renderApoios() {
```

```
  const c = document.getElementById('apoios-container');
```

```

    let h = `<table
class="data-table"><thead><tr><th>Nó</th><th>Tipo</th></tr></thead><tbody>`;
    apoios.forEach((a,i) => {
        let opts = nos.map((_,n)=>`<option value="${n}"
${a.no===n?'selected':''}>N${n}</option>`).join('');
        h += `<tr>
            <td><select onchange="apoios[${i}].no+=this.value"
style="padding:3px 5px">${opts}</select></td>
            <td><select onchange="apoios[${i}].tipo=this.value"
style="padding:3px 5px">
                <option value="fixo" ${a.tipo==='fixo'?'selected':''}>Fixo
(Rx,Ry)</option>
                <option value="rolete_x"
${a.tipo==='rolete_x'?'selected':''}>Rolete ↕ (Ry)</option>
                <option value="rolete_y"
${a.tipo==='rolete_y'?'selected':''}>Rolete ↔ (Rx)</option>
            </select></td>
        </tr>`;
    });
    h += `</tbody></table>`;
    c.innerHTML = h;
}

function renderCargas() {
    const c = document.getElementById('cargas-container');
    let h = `<table
class="data-table"><thead><tr><th>Nó</th><th>Fx (kN)</th><th>Fy (kN)</th>
</tr></thead><tbody>`;
    cargas.forEach((cg,i) => {
        let opts = nos.map((_,n)=>`<option value="${n}"
${cg.no===n?'selected':''}>N${n}</option>`).join('');
        h += `<tr>
            <td><select onchange="cargas[${i}].no+=this.value"
style="padding:3px 5px">${opts}</select></td>
            <td><input type="number" step="1" value="${cg.fx}"
onchange="cargas[${i}].fx+=this.value" style="padding:3px 5px"></td>
            <td><input type="number" step="1" value="${cg.fy}"
onchange="cargas[${i}].fy+=this.value" style="padding:3px 5px"></td>
        </tr>`;
    });
    h += `</tbody></table>`;
    c.innerHTML = h;
}

```

```

}

function addNo() { nos.push({x:0,y:0}); renderNos(); }
function removeNo() { if(nos.length>1) nos.pop(); renderNos(); }
function addBarra() { if(nos.length>=2) barras.push({i:0,j:1});
renderBarras(); }
function removeBarra() { if(barras.length>0) barras.pop();
renderBarras(); }
function addApoio() { apoios.push({no:0,tip:'fixo'}); renderApoios();
}
function removeApoio() { if(apoios.length>0) apoios.pop();
renderApoios(); }
function addCarga() { cargas.push({no:0,fx:0,fy:0}); renderCargas(); }
function removeCarga() { if(cargas.length>0) cargas.pop();
renderCargas(); }

// === ANÁLISE ESTRUTURAL (Método da Rigidez Direta) ===
function analisar() {
  const msg = document.getElementById('status-msg');
  try {
    const nNos = nos.length;
    const nBarras = barras.length;
    const nDOF = 2 * nNos;

    // Verificação isostática
    let nReacoes = 0;
    apoios.forEach(a => { nReacoes += (a.tip==='fixo') ? 2 : 1; });
    const grau = nBarras + nReacoes - 2*nNos;

    if (grau < 0) { msg.innerHTML = `<div class="alert alert-danger">⚠
Estrutura hipostática (grau ${grau}): insuficientemente
restringida.</div>`; return; }
    if (grau > 0) { msg.innerHTML = `<div class="alert alert-danger">⚠
Estrutura hiperestática (grau ${grau}): este solver resolve apenas
isostáticas. Adicione barras ou remova restrições.</div>`; return; }

    // Montar vetor de forças externas
    const F = new Array(nDOF).fill(0);
    cargas.forEach(cg => {
      F[2*cg.no] += cg.fx;
      F[2*cg.no+1] += cg.fy;
    });
  }
}

```



```

// Índices livres (DOF) e restringidos (apoios)
const fixedDOF = new Set();
apoios.forEach(a => {
    if(a.tipo==='fixo') { fixedDOF.add(2*a.no);
fixedDOF.add(2*a.no+1); }
    else if(a.tipo==='rolete_x') { fixedDOF.add(2*a.no+1); }
    else if(a.tipo==='rolete_y') { fixedDOF.add(2*a.no); }
});

const freeDOF = [];
for(let i=0;i<nDOF;i++) { if(!fixedDOF.has(i)) freeDOF.push(i); }

// Matriz de rigidez global
const K = Array.from({length:nDOF}, ()=>new Array(nDOF).fill(0));
const comprimentos = [], angulos = [];

barras.forEach((b,k) => {
    const ni = b.i, nj = b.j;
    const dx = nos[nj].x - nos[ni].x;
    const dy = nos[nj].y - nos[ni].y;
    const L = Math.sqrt(dx*dx+dy*dy);
    comprimentos.push(L);
    angulos.push(Math.atan2(dy,dx));
    const c = dx/L, s = dy/L;
    const EA_L = 1.0; // normalizado (forças em função de F, não de
EA)

    // Matriz de rigidez local da barra
    const ke =
[[c*c,c*s,-c*c,-c*s],[c*s,s*s,-c*s,-s*s],[-c*c,-c*s,c*c,c*s],[-c*s,-s*s
,c*s,s*s]];

    const dofs = [2*ni,2*ni+1,2*nj,2*nj+1];
    for(let a=0;a<4;a++) for(let bb=0;bb<4;bb++) K[dofs[a]][dofs[bb]]
+= ke[a][bb];
});

// Extrair submatriz livre
const Kff = freeDOF.map(i => freeDOF.map(j => K[i][j]));
const Ff = freeDOF.map(i => F[i]);

// Resolver Kff * U = Ff usando eliminação gaussiana
const U_free = gaussianElimination(Kff, Ff);

```

```

    if(!U_free) { msg.innerHTML = `<div class="alert alert-danger">⚠
Sistema singular – verifique apoios e topologia da estrutura.</div>`;
return; }

const U = new Array(nDOF).fill(0);
freeDOF.forEach((dof,i) => { U[dof] = U_free[i]; });

// Calcular forças nas barras
const forcas = barras.map((b,k) => {
    const ni = b.i, nj = b.j;
    const dx = nos[nj].x - nos[ni].x;
    const dy = nos[nj].y - nos[ni].y;
    const L = comprimentos[k];
    const c = dx/L, s = dy/L;
    const u = [U[2*ni],U[2*ni+1],U[2*nj],U[2*nj+1]];
    const F_barra = (c*u[2]+s*u[3]-c*u[0]-s*u[1]);
    return F_barra;
});

// Calcular reações
const R = new Array(nDOF).fill(0);
for(let i=0;i<nDOF;i++) {
    for(let j=0;j<nDOF;j++) R[i] += K[i][j]*U[j];
    R[i] -= F[i];
}

const reacoes = [];
apoios.forEach(a => {
    if(a.tipo==='fixo') {
        reacoes.push({no:a.no,dir:'x',valor:R[2*a.no]});
        reacoes.push({no:a.no,dir:'y',valor:R[2*a.no+1]});
    } else if(a.tipo==='rolete_x') {
        reacoes.push({no:a.no,dir:'y',valor:R[2*a.no+1]});
    } else if(a.tipo==='rolete_y') {
        reacoes.push({no:a.no,dir:'x',valor:R[2*a.no]});
    }
});

resultados = { forcas, reacoes, comprimentos, grau, U, nNos,
nBarras };
mostrarResultados();
mostrarVisualizacao();

```

```

    msg.innerHTML = `<div class="alert alert-success">✓ Análise
concluída com sucesso!</div>`;
    showTab('resultados');
  } catch(e) {
    msg.innerHTML = `<div class="alert alert-danger">Erro:
${e.message}</div>`;
    console.error(e);
  }
}

function gaussianElimination(A, b) {
  const n = b.length;
  const M = A.map((row, i) => [...row, b[i]]);
  for(let col=0; col<n; col++) {
    let maxRow=col;
    for(let row=col+1; row<n; row++)
if (Math.abs(M[row][col])>Math.abs(M[maxRow][col])) maxRow=row;
    [M[col], M[maxRow]] = [M[maxRow], M[col]];
    if(Math.abs(M[col][col])<1e-12) return null;
    for(let row=col+1; row<n; row++) {
      const f=M[row][col]/M[col][col];
      for(let k=col; k<=n; k++) M[row][k]-=f*M[col][k];
    }
  }
  const x=new Array(n).fill(0);
  for(let i=n-1; i>=0; i--) {
    x[i]=M[i][n]/M[i][i];
    for(let k=i-1; k>=0; k--) M[k][n]-=M[k][i]*x[i];
  }
  return x;
}

// === MOSTRAR RESULTADOS ===
function mostrarResultados() {
  if(!resultados) return;
  const { forcas, reacoes, complimentos, grau, nNos, nBarras } =
resultados;
  const maxF = Math.max(...forcas.map(f=>Math.abs(f)));

  let html = `
<div class="stat-grid">
  <div class="stat-card"><div class="stat-val
num-strong">${nNos}</div><div class="stat-lbl">Nós</div></div>

```

```

    <div class="stat-card"><div class="stat-val
num-strong">${nBarras}</div><div class="stat-lbl">Barras</div></div>
    <div class="stat-card"><div class="stat-val num-strong"
style="color:var(--color-text-success)">ISO</div><div
class="stat-lbl">Tipo de Estrutura</div></div>
    <div class="stat-card"><div class="stat-val
num-strong">${maxF.toFixed(2)}</div><div class="stat-lbl">Força Máx.
(kN·m)</div></div>
  </div>

  <div class="section">
    <div class="section-label">Forças Internas nas Barras</div>
    <table class="data-table">
      <thead><tr><th>Barra</th><th>Nó i → j</th><th>Comprimento
(m)</th><th>Força
(kN·m)</th><th>Tipo</th><th>Diagrama</th></tr></thead>
      <tbody>`;

  forcas.forEach((f,k) => {
    const b = barras[k];
    const tipo = Math.abs(f)<0.001 ? 'zero' : (f>0 ? 'tracao' :
'compressao');
    const pill = tipo==='zero' ? '<span class="pill
pill-z">NULA</span>' : (tipo==='tracao'? '<span class="pill
pill-t">TRAÇÃO</span>': '<span class="pill pill-c">COMPRESSÃO</span>');
    const pct = maxF > 0 ? Math.abs(f)/maxF*100 : 0;
    const cor = tipo==='tracao' ? '#3b82f6' : tipo==='compressao' ?
'#ef4444' : '#9ca3af';
    html += `<tr>
      <td><strong>B${k}</strong></td>
      <td
style="color:var(--color-text-secondary)">N${b.i}→N${b.j}</td>
      <td>${comprimentos[k].toFixed(2)}</td>
      <td class="num-strong">${f.toFixed(4)}</td>
      <td>${pill}</td>
      <td><div
style="background:${cor};height:6px;border-radius:2px;width:${pct}%;min
-width:2px;max-width:80px"></div></td>
    </tr>`;
  });

  html += `</tbody></table></div>

```

```

<div class="section">
  <div class="section-label">Reações nos Apoios</div>
  <table class="data-table">
    <thead><tr><th>Nó</th><th>Direção</th><th>Reação
(kN)</th><th>Sentido</th></tr></thead>
    <tbody>`;

    reacoes.forEach(r => {
      const seta = r.valor >= 0 ? (r.dir==='x'? '→': '↑') :
(r.dir==='x'? '←': '↓');
      html += `<tr>
        <td><strong>N${r.no}</strong></td>
        <td>${r.dir.toUpperCase()}</td>
        <td class="num-strong">${r.valor.toFixed(4)}</td>
        <td style="font-size:16px">${seta}</td>
      </tr>`;
    });

    html += `</tbody></table></div>

<div class="section">
  <div class="section-label">Verificação de Equilíbrio</div>`;

  let sFx=0, sFy=0;
  cargas.forEach(c=>{sFx+=c.fx; sFy+=c.fy;});
  reacoes.forEach(r=>{ if(r.dir==='x') sFx+=r.valor; else sFy+=r.valor;
});

  const ok_x = Math.abs(sFx)<0.01, ok_y = Math.abs(sFy)<0.01;
  html += `<table class="data-table">

<thead><tr><th>Verificação</th><th>Valor</th><th>Status</th></tr></thead>

  <tbody>
    <tr><td>ΣFx = 0</td><td>${sFx.toFixed(6)} kN</td><td>${ok_x?'<span
class="pill pill-t">✓ OK</span>': '<span class="pill pill-c">✗
ERRO</span>'}</td></tr>
    <tr><td>ΣFy = 0</td><td>${sFy.toFixed(6)} kN</td><td>${ok_y?'<span
class="pill pill-t">✓ OK</span>': '<span class="pill pill-c">✗
ERRO</span>'}</td></tr>
  </tbody></table></div>`;

  document.getElementById('results-content').innerHTML = html;

```

```

}

// === VISUALIZAÇÃO ===
function mostrarVisualizacao() {
  if(!resultados || !nos.length) return;
  const { forcas } = resultados;

  let html = `
    <div class="section">
      <div class="section-label">Diagrama Estrutural</div>
      <div class="canvas-wrap"><canvas id="truss-canvas" width="640"
height="340"></canvas></div>
      <div class="legend">
        <div class="legend-item"><div class="legend-dot"
style="background:#3b82f6"></div>Tração</div>
        <div class="legend-item"><div class="legend-dot"
style="background:#ef4444"></div>Compressão</div>
        <div class="legend-item"><div class="legend-dot"
style="background:#9ca3af"></div>Força nula</div>
        <div class="legend-item"><div class="legend-dot"
style="background:#f59e0b;height:10px;width:10px;border-radius:50%"></d
iv>Apoio fixo</div>
      </div>
    </div>

    <div class="section" style="margin-top:16px">
      <div class="section-label">Gráfico de Forças por Barra</div>
      <div class="canvas-wrap"><canvas id="bar-canvas" width="640"
height="220"></canvas></div>
    </div>`;

  document.getElementById('viz-content').innerHTML = html;

  setTimeout(() => {
    drawTruss();
    drawBarChart();
  }, 50);
}

function getCanvasColors() {
  const s = getComputedStyle(document.documentElement);
  const isDark = window.matchMedia('(prefers-color-scheme:
dark)').matches;

```

```

return {
  bg: isDark ? '#1a1a1a' : '#f8f8f6',
  grid: isDark ? 'rgba(255,255,255,0.06)' : 'rgba(0,0,0,0.06)',
  text: isDark ? '#c2c0b6' : '#3d3d3a',
  muted: isDark ? '#6b6a62' : '#9ca3af',
  node: isDark ? '#3b82f6' : '#2563eb',
  support: '#f59e0b',
  load: '#8b5cf6',
};
}

function worldToCanvas(x, y, margin, scaleX, scaleY, minX, minY,
canvasH) {
  return [(x - minX)*scaleX + margin, canvasH - margin - (y -
minY)*scaleY];
}

function drawTruss() {
  const canvas = document.getElementById('truss-canvas');
  if(!canvas) return;
  const ctx = canvas.getContext('2d');
  const W = canvas.width, H = canvas.height;
  const margin = 50;
  const col = getCanvasColors();
  const { forcas } = resultados;

  ctx.clearRect(0,0,W,H);
  ctx.fillStyle = col.bg;
  ctx.fillRect(0,0,W,H);

  if(!nos.length) return;
  const xs = nos.map(n=>n.x), ys = nos.map(n=>n.y);
  const minX=Math.min(...xs), maxX=Math.max(...xs),
minY=Math.min(...ys), maxY=Math.max(...ys);
  const rangeX = maxX-minX || 1, rangeY = maxY-minY || 1;
  const scaleX = (W-2*margin)/rangeX, scaleY = (H-2*margin)/rangeY;
  const scale = Math.min(scaleX, scaleY);

  const toC = (x,y) => {
    const cx = (x-minX)*scale + margin + (W-2*margin-(rangeX*scale))/2;
    const cy = H - margin - (y-minY)*scale -
(H-2*margin-(rangeY*scale))/2;
    return [cx,cy];
  }

```

```

};

// Grade
ctx.strokeStyle = col.grid; ctx.lineWidth = 0.5;
for(let gx=Math.floor(minX); gx<=Math.ceil(maxX); gx++) {
  const [cx1,cy1]=toC(gx,minY-0.2), [cx2,cy2]=toC(gx,maxY+0.2);
  ctx.beginPath(); ctx.moveTo(cx1,cy1); ctx.lineTo(cx2,cy2);
ctx.stroke();
}
for(let gy=Math.floor(minY); gy<=Math.ceil(maxY); gy++) {
  const [cx1,cy1]=toC(minX-0.2,gy), [cx2,cy2]=toC(maxX+0.2,gy);
  ctx.beginPath(); ctx.moveTo(cx1,cy1); ctx.lineTo(cx2,cy2);
ctx.stroke();
}

const maxF = Math.max(...forcas.map(f=>Math.abs(f)), 0.001);

// Barras
barras.forEach((b,k) => {
  const f = forcas[k];
  const [x1,y1] = toC(nos[b.i].x, nos[b.i].y);
  const [x2,y2] = toC(nos[b.j].x, nos[b.j].y);
  const pct = Math.abs(f)/maxF;
  const lw = 1.5 + pct*5;

  let cor = '#9ca3af';
  if(f > 0.001) cor = `rgba(59,130,246,${0.4+pct*0.6})`;
  else if(f < -0.001) cor = `rgba(239,68,68,${0.4+pct*0.6})`;

  ctx.strokeStyle = cor; ctx.lineWidth = lw;
  ctx.beginPath(); ctx.moveTo(x1,y1); ctx.lineTo(x2,y2);
ctx.stroke();

  // Rótulo da força
  const mx=(x1+x2)/2, my=(y1+y2)/2;
  ctx.fillStyle = cor; ctx.font = '9px monospace'; ctx.textAlign =
'center';
  ctx.fillText(`${f.toFixed(2)}`, mx, my-5);
});

// Apoios
apoios.forEach(a => {
  const [cx,cy] = toC(nos[a.no].x, nos[a.no].y);

```



```

    ctx.fillStyle = col.support;
    if(a.tipo==='fixo') {
        ctx.beginPath(); ctx.moveTo(cx,cy); ctx.lineTo(cx-12,cy+18);
ctx.lineTo(cx+12,cy+18); ctx.closePath(); ctx.fill();
        ctx.strokeStyle = col.support; ctx.lineWidth = 1.5;
        ctx.beginPath(); ctx.moveTo(cx-15,cy+18);
ctx.lineTo(cx+15,cy+18); ctx.stroke();
    } else {
        ctx.beginPath(); ctx.moveTo(cx,cy); ctx.lineTo(cx-10,cy+15);
ctx.lineTo(cx+10,cy+15); ctx.closePath(); ctx.fill();
        ctx.beginPath(); ctx.arc(cx,cy+19,4,0,Math.PI*2); ctx.fill();
    }
});

// Cargas
cargas.forEach(cg => {
    const [cx,cy] = toC(nos[cg.no].x, nos[cg.no].y);
    ctx.strokeStyle = col.load; ctx.fillStyle = col.load; ctx.lineWidth
= 2;
    if(Math.abs(cg.fy) > 0.001) {
        const dir = cg.fy > 0 ? -1 : 1;
        const len = 30;
        ctx.beginPath(); ctx.moveTo(cx, cy + dir*8); ctx.lineTo(cx, cy +
dir*(8+len)); ctx.stroke();
        ctx.beginPath(); ctx.moveTo(cx,cy+dir*8);
ctx.lineTo(cx-6,cy+dir*20); ctx.lineTo(cx+6,cy+dir*20);
ctx.closePath(); ctx.fill();
        ctx.font = '9px monospace'; ctx.textAlign = 'center';
        ctx.fillText(`${Math.abs(cg.fy)}kN`, cx, cy + dir*(8+len+10));
    }
    if(Math.abs(cg.fx) > 0.001) {
        const dir = cg.fx > 0 ? 1 : -1;
        ctx.beginPath(); ctx.moveTo(cx+dir*8,cy);
ctx.lineTo(cx+dir*38,cy); ctx.stroke();
        ctx.beginPath(); ctx.moveTo(cx+dir*8,cy);
ctx.lineTo(cx+dir*20,cy-6); ctx.lineTo(cx+dir*20,cy+6);
ctx.closePath(); ctx.fill();
    }
});

// Nós
nos.forEach((n,i) => {
    const [cx,cy] = toC(n.x, n.y);

```

```

    ctx.fillStyle = col.node;
    ctx.beginPath(); ctx.arc(cx,cy,6,0,Math.PI*2); ctx.fill();
    ctx.fillStyle = '#fff';
    ctx.beginPath(); ctx.arc(cx,cy,3.5,0,Math.PI*2); ctx.fill();
    ctx.fillStyle = col.text; ctx.font = 'bold 10px monospace';
ctx.textAlign = 'center';
    ctx.fillText(`N${i}`, cx, cy-10);
    ctx.fillStyle = col.muted; ctx.font = '8px monospace';
    ctx.fillText(`(${n.x},${n.y})`, cx, cy+18);
  });
}

function drawBarChart() {
  const canvas = document.getElementById('bar-canvas');
  if(!canvas) return;
  const ctx = canvas.getContext('2d');
  const W = canvas.width, H = canvas.height;
  const col = getCanvasColors();
  const { forcas } = resultados;

  ctx.clearRect(0,0,W,H);
  ctx.fillStyle = col.bg;
  ctx.fillRect(0,0,W,H);

  const n = forcas.length;
  const marginL=55, marginR=20, marginT=20, marginB=50;
  const cW = W-marginL-marginR, cH = H-marginT-marginB;
  const maxF = Math.max(...forcas.map(f=>Math.abs(f)), 0.001);

  // Eixo zero
  const zeroY = marginT + cH/2;
  ctx.strokeStyle = col.muted; ctx.lineWidth = 0.5;
  ctx.beginPath(); ctx.moveTo(marginL,zeroY);
ctx.lineTo(W-marginR,zeroY); ctx.stroke();

  // Grade horizontal
  [-1,-0.5,0,0.5,1].forEach(v => {
    const y = marginT + cH*(0.5 - v*0.5);
    ctx.strokeStyle = col.grid; ctx.lineWidth = 0.5;
    ctx.beginPath(); ctx.moveTo(marginL,y); ctx.lineTo(W-marginR,y);
ctx.stroke();
    ctx.fillStyle = col.muted; ctx.font = '9px monospace';
ctx.textAlign = 'right';

```

```

    ctx.fillText((v*maxF).toFixed(1), marginL-4, y+3);
  });

  const bw = Math.max(10, Math.min(50, cW/n - 6));
  const step = cW/n;

  forcas.forEach((f,k) => {
    const x = marginL + k*step + step/2;
    const pct = f/maxF;
    const barH = Math.abs(pct)*cH*0.5;
    const barY = f>=0 ? zeroY-barH : zeroY;
    const cor = f>0.001 ? '#3b82f6' : f<-0.001 ? '#ef4444' : '#9ca3af';

    ctx.fillStyle = cor;
    ctx.fillRect(x-bw/2, barY, bw, barH || 2);

    ctx.fillStyle = col.text; ctx.font = '9px monospace'; ctx.textAlign
= 'center';
    ctx.fillText(`B${k}`, x, H-marginB+12);
    ctx.fillStyle = cor; ctx.font = '8px monospace';
    ctx.fillText(f.toFixed(2), x, f>=0 ? barY-4 : barY+barH+11);
  });

  // Eixos
  ctx.strokeStyle = col.muted; ctx.lineWidth = 1;
  ctx.beginPath(); ctx.moveTo(marginL,marginT);
ctx.lineTo(marginL,H-marginB); ctx.lineTo(W-marginR,H-marginB);
ctx.stroke();

  ctx.fillStyle = col.text; ctx.font = '10px monospace'; ctx.textAlign
= 'center';
  ctx.fillText('Barras', W/2, H-4);
  ctx.save(); ctx.translate(12, H/2); ctx.rotate(-Math.PI/2);
  ctx.fillText('Força (kN·m)', 0, 0); ctx.restore();
}

// === TABS ===
function showTab(name) {
  document.querySelectorAll('.tab').forEach(t =>
t.classList.remove('active'));
  document.querySelectorAll('.tab-content').forEach(t =>
t.classList.remove('active'));
  document.getElementById('tab-'+name).classList.add('active');
}

```

```

document.querySelectorAll('.tab').forEach(t => {
    if(t.getAttribute('onclick').includes("'" + name + "'"))
t.classList.add('active');
});
if(name==='visualizacao' && resultados) { setTimeout(=>{
drawTruss(); drawBarChart(); }, 80); }
}

// === INIT ===
loadPreset('warren');
</script>
</body>
</html>

```

Gemini:

```

import numpy as np
import matplotlib.pyplot as plt

```

```

# =====
# 1. DEFINIÇÃO DA ESTRUTURA (Edite aqui)
# =====

```

```

# Coordenadas dos nós [x, y]
nos = np.array([
    [0, 0], # Nó 0
    [2, 2], # Nó 1
    [4, 0] # Nó 2
])

```

```

# Conexões das barras [nó_inicial, nó_final]
barras = [
    [0, 1], # Barra 0
    [1, 2], # Barra 1
    [0, 2] # Barra 2
]

```

```

# Cargas aplicadas [nó, força_x, força_y]
# Exemplo: Carga de 1000N para baixo no nó 1
cargas = [
    [1, 0, -1000]
]

```

```
]
```

```
# Apoios [nó, direção] -> 'x' para horizontal, 'y' para vertical
```

```
# Para ser isostática, precisamos de 3 reações
```

```
apoios = [
```

```
    [0, 'x'], [0, 'y'], # Apoio fixo no nó 0
```

```
    [2, 'y']           # Apoio móvel no nó 2
```

```
]
```

```
# =====
```

```
# 2. PROCESSAMENTO (Cálculos Matemáticos)
```

```
# =====
```

```
num_nos = len(nos)
```

```
num_barras = len(barras)
```

```
num_reacoes = len(apoios)
```

```
num_equacoes = 2 * num_nos
```

```
# Matriz Global A e Vetor de Cargas B
```

```
A = np.zeros((num_equacoes, num_equacoes))
```

```
B = np.zeros(num_equacoes)
```

```
# Preenchendo barras na matriz (Componentes de força)
```

```
for i, (n1, n2) in enumerate(barras):
```

```
    dx = nos[n2][0] - nos[n1][0]
```

```
    dy = nos[n2][1] - nos[n1][1]
```

```
    L = np.sqrt(dx**2 + dy**2)
```

```
    cos, sen = dx/L, dy/L
```

```
    # Nó Inicial
```

```
    A[2*n1, i] = cos
```

```
    A[2*n1+1, i] = sen
```

```
    # Nó Final
```

```
    A[2*n2, i] = -cos
```

```
    A[2*n2+1, i] = -sen
```

```
# Preenchendo reações de apoio na matriz
```

```
for i, (no, direcao) in enumerate(apoios):
```

```
    coluna = num_barras + i
```

```
    if direcao == 'x':
```

```
        A[2*no, coluna] = 1
```

```
    else:
```

```
        A[2*no+1, coluna] = 1
```

```

# Preenchendo vetor de cargas
for no, fx, fy in cargas:
    B[2*no] = -fx
    B[2*no+1] = -fy

# Resolvendo o sistema linear Ax = B
try:
    resultado = np.linalg.solve(A, B)
    forcas_barras = resultado[:num_barras]
    reacoes = resultado[num_barras:]
except np.linalg.LinAlgError:
    print("ERRO: A estrutura é instável (hipostática) ou mal definida!")
    resultado = None

# =====
# 3. VISUALIZAÇÃO DOS RESULTADOS
# =====

if resultado is not None:
    print(f'{"BARRA":<10} | {"FORÇA (N)":<15} | {"ESTADO"}')
    print("-" * 40)
    for i, f in enumerate(forcas_barras):
        estado = "TRAÇÃO" if f > 0.1 else "COMPRESSÃO" if f < -0.1 else "NULA"
        print(f'{"i":<10} | {"f":>15.2f} | {estado}')

# Gráfico
plt.figure(figsize=(8, 5))
for i, (n1, n2) in enumerate(barras):
    cor = 'red' if forcas_barras[i] < -0.1 else 'blue' if forcas_barras[i] > 0.1 else
'black'
    plt.plot([nos[n1][0], nos[n2][0]], [nos[n1][1], nos[n2][1]], color=cor, linewidth=2,
marker='o')

    # Texto da força no meio da barra
    mx, my = (nos[n1][0]+nos[n2][0])/2, (nos[n1][1]+nos[n2][1])/2
    plt.text(mx, my, f'{forcas_barras[i]:.1f}N', fontsize=9, fontweight='bold')

plt.title("Análise Estrutural: Azul (Tração) | Vermelho (Compressão)")
plt.xlabel("X (m)")
plt.ylabel("Y (m)")
plt.grid(True, linestyle='--')
plt.show()

```

Comparando as LLMs:

Gemini escolheu o método dos nós clássico — monta um sistema linear $Ax = B$ onde as incógnitas são as forças nas barras e as reações, tudo de uma vez. É o método mais direto para estruturas isostáticas e o código mais curto e legível dos três.

ChatGPT usou o Método dos Elementos Finitos (FEM) completo — monta a matriz de rigidez global K , aplica condições de contorno reduzindo o sistema, e resolve deslocamentos primeiro. A vantagem é que calcula deformações reais com $E \cdot A$, e a abordagem é a mesma de softwares como SAP2000 ou ANSYS.

Claude não é um script — é uma aplicação web interativa que roda no browser, sem Python. O motor de cálculo usa rigidez direta com eliminação gaussiana em JavaScript. A grande diferença está na usabilidade: você edita a estrutura pela interface gráfica, tem exemplos pré-carregados (Warren, Pratt, ponte), e a visualização atualiza em tempo real com barras proporcionais às forças.

Cada um tem seu lugar: Gemini para estudar o método, ChatGPT para rigor físico, Claude para uso prático sem configurar ambiente Python.

Gemini

Parece mais simples

Menos código, mais fácil de olhar.

ChatGPT

Parece mais completo

Tinha mais coisa, mais organizado em partes. Deu a impressão de ser mais sério, mas também mais difícil de mexer.

Claude

Parece o mais "pronto"

Não precisava saber programar pra usar. Tinha uma interface bonitinha com exemplos já prontos. Dava pra testar sem entender nada.